

Are all BSDs created equally?

A survey of BSD kernel vulnerabilities.

Ilja van Sprundel <ivansprundel@ioactive.com>

Who Am I

- Ilja van Sprundel
- ivansprundel@ioactive.com
- Director of Penetration Testing at IOActive
- Pen test
- Code review
- Break stuff for fun and profit 😊

Outline/Agenda

- Intro
- Data!
 - vulnerabilities over the years
- Test by audit
 - Common attack surface
 - Somewhat less common attack surface
- Some results / conclusions

What is this talk about?

- BSD kernel vulnerabilities
 - Comparison
 - Between different BSD flavors
- Audience
 - Low level security enthusiasts
 - UNIX/BSD geeks
 - I suspect Linux folks might enjoy this too 😊
 - Curious people that like to poke around in OS internals
- Knowledge
 - Some basic knowledge of UNIX / BSD internals

Standing on the shoulders of giants

- Previous interesting BSD kernel security research by:
 - Silvio
 - the noir
 - Esa Etelavuori
 - Patroklos (argp) Argyroudis
 - Christer Oberg
 - Joel Eriksson
 - Clement Lecigne

Re: Theo gave an interview to Forbes Mag. about Linux

Theo de Raadt | Fri, 17 Jun 2005 09:30:05 -0700

> On Fri, Jun 17, 2005 at 04:48:31PM +0200, J. Lievisse Adriaanse wrote:

...

If the Linux people actually cared about Quality, as we do, they would not have had as many localhost kernel security holes in the last year.

How many is it... 20 so far?

intro

Really? Got Data?

- Somehow that statement has always been stuck in my head
- Is it true?
- Can we look at some data ?

Source: <https://www.cvedetails.com/product/47/Linux-Linux-Kernel.html>

[Linux](#) » [Linux Kernel](#) : Vulnerability Statistics

Vulnerability Trends Over Time

Year	# of Vulnerabilities	DoS	Code Execution	Overflow	Memory Corruption	Sql Injection	XSS	Directory Traversal	Http Response Splitting	Bypass something	Gain Information	Gain Privileges	CSRF	File Inclusion	# of exploits
1999	19	7		3						1		2			
2000	5	3										1			
2001	23	7								4		3			
2002	15	3		1						1	1				
2003	19	8		2						1	3	4			
2004	51	20	5	12							5	12			
2005	133	90	19	19	1					6	5	7			
2006	90	61	5	7	7			2		5	3	3			
2007	63	41	2	8						3	7	7			1
2008	70	44	3	17	4					4	6	10			4
2009	105	66	2	22	7					8	11	22			4
2010	124	67	3	16	7					8	30	14			5
2011	83	62	1	21	10					1	21	9			1
2012	115	83	4	25	10					6	19	11			
2013	189	101	6	41	13					11	57	26			7
2014	133	89	8	21	10					11	30	20			10
2015	86	55	6	15	4					11	10	17			
2016	217	153	5	38	18					12	35	52			1
2017	346	80	165	29	14					9	74	23			
Total	1886	1040	234	297	105			2		102	317	243			33
% Of All		55.1	12.4	15.7	5.6	0.0	0.0	0.1	0.0	5.4	16.8	12.9	0.0	0.0	

Data!

- Goes from current back to 1999 for Linux kernel vulnerabilities
- Cvedetails.com doesn't seem to provide data for OBSD/NBSD/FBSD
- Manually grab it from
 - <https://www.freebsd.org/security/advisories.html>
 - <http://netbsd.org/support/security/advisory.html>
 - https://www.openbsd.org/errata*.html

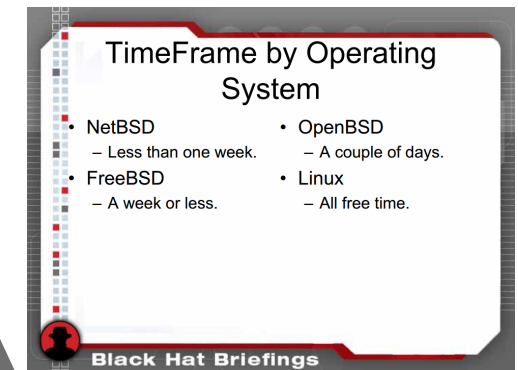
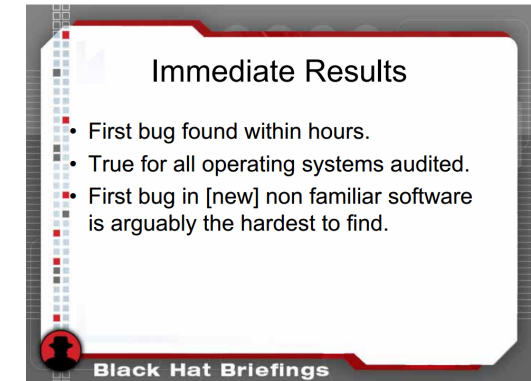
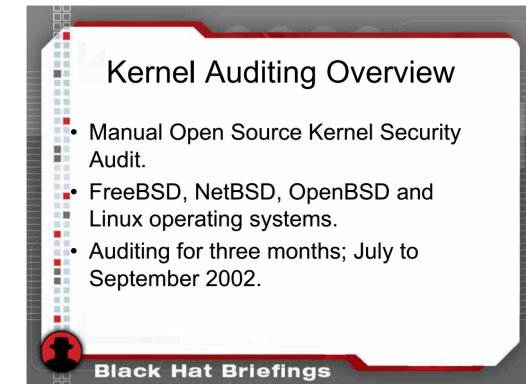
BSD kernel vulnerabilities over the years

	FreeBSD	NetBSD	OpenBSD
1999	3	8	10
2000	8	4	15
2001	6	7	13
2002	11	6	11
2003	7	3	8
2004	8	5	13
2005	11	8	4
2006	9	15	6
2007	1	4	9
2008	8	6	7
2009	5	1	4
2010	3	6	8
2011	1	2	3
2012	2	1	0
2013	8	8	6
2014	7	6	10
2015	7	2	6
2016	12	1	17
2017	1	3	13
Total	118	96	163

- Looking at these numbers, that was an astute observation by Theo.
 - 20 was a very low estimate
- But are these numbers on equal footing?
- Many eyeballs?
 - Yea, yea, I know But is there some truth to it in this case?

Test by audit!

- Silvio Cesare did some interesting work in ~2002 that gives some answers
- <https://www.blackhat.com/presentations/bh-usa-03/bh-us-03-cesare.pdf>
- His results seem to indicate there isn't really that much of a quality difference. However:
 - that was well over a decade ago.
 - Have things changed?
 - Time spend on the BSDs was only a couple of days compared to Linux
 - If more time would've been spend, would more bugs have been found?
 - bugs are mostly int overflows and info leaks
 - Other kinds of issues that can 'easily' be found ?



Test by Audit redux.

- Spend April-May-June auditing BSD source code.
- Asked myself, “where would the bugs be?”
- Attack surface
 - Very common
 - Syscalls
 - TCP/IP stack
 - Somewhat less common (in ascending order, more or less)
 - Drivers (ioctl interface)
 - compat code
 - Trap handlers
 - Filesystems
 - Other networking (BT, wifi, IrDA)



Syscalls

Attack surface entrypoint

- The obvious attack surface
- Syscalls are how userland gets anything done from kernel
- Hundreds of them
 - FreeBSD: ~550
 - OpenBSD: ~330
 - NetBSD: ~480
- Assumption: given that they're obvious, and well tested, less likely to contain security bugs

```
/*
 * System call numbers.
 *
 * DO NOT EDIT-- this file is automatically generated.
 * $FreeBSD: releng/11.0/sys/sys/syscall.h 303858 2016-08-08 21:19:57Z bdwery $
 * created from FreeBSD: stable/11/sys/kern/syscalls.master 303854 2016-08-08 20:23:11Z bdwery
 */

#define SYS_syscall    0
#define SYS_exit      1
#define SYS_fork       2
#define SYS_read       3
#define SYS_write      4
#define SYS_open       5
#define SYS_close      6
#define SYS_wait4      7

#define SYS_link       9      /* 8 is old creat */
#define SYS_unlink    10
#define SYS_chdir     12      /* 11 is obsolete execv */
#define SYS_fchdir    13
#define SYS_mknod     14
#define SYS_chmod     15
#define SYS_chown     16
#define SYS_break     17

#define SYS_getpid    20
#define SYS_mount     21
#define SYS_unmount   22
#define SYS_setuid    23
#define SYS_getuid    24
#define SYS_geteuid   25
#define SYS_ptrace    26
#define SYS_recvmmsg  27
#define SYS_sendmsg   28
#define SYS_recvfrom  29
#define SYS_accept    30
#define SYS_getpeername 31
/usr/include/sys/syscall.h: unmodified, readonly: line 1
```

```

int
sys_sendsyslog(struct
{
    struct sys_

    } */ *uap =
    int error;
    static int d
...
    error = dos
    SCARG(uap, nbyte),
    SCARG(u
...
    return (err
}

```

```

int
dosendsyslog(struct proc *p, const char *buf, size_t nbyte, int flags,
enum uio_seg sflg)
{
...

```

```

    struct iovec aiov;
    struct uio auio;
    size_t i, len;
...

```

```

    aiov.iov_base = (char *)buf;
    aiov.iov_len = nbyte; ← user controlled size_t. never capped anywhere
...

```

```

    auio.uio_res

```

```

    len = auio.uio

```

```

    if (fp) {
...

```

```

    } else if (con
...

```

```

    } else {
...

```

```

    }
...
}

```

```

$ make syslog
cc -O2 -pipe -o syslog syslog.c
$ ./syslog

```

```

panic: malloc: allocation too large, type = 127, size = 4294967295

```

```

Stopped at Debugger+0x9: leave
TID    PID    UID    PRFLAGS    PFLAGS    CPU    COMMAND
*13393 13393 1000    0x3        0        0    syslog

```

```

Debugger() at Debugger+0x9
panic() at panic+0xfe
malloc() at malloc+0x621
dosendsyslog() at dosendsyslog+0x3cf
sys_sendsyslog() at sys_sendsyslog+0xbe
syscall() at syscall+0x197
--- syscall (number 112) ---

```

```

end of kernel
end trace frame: 0x7f7ffffd8e30, count: 9
0x108c4ac1604a:

```

```

http://www.openbsd.org/ddb.html describes the minimum info required in bug
reports. Insufficient info makes it difficult to find and fix bugs.
ddb>

```

Sample bug

- sendsyslog system call
- OpenBSD 6.1
 - Been there since OpenBSD 6.0
- Unbound length passed to malloc() from userland
- Will trigger a kernel panic


```

int
sys_kldstat(struct thread *td, struct kldstat_args *uap)
{
    struct kld_file_stat stat;
    int error, version;

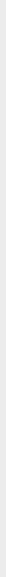
    /*
     * Check the version of the user's structure.
     */
    if ((error = copyin(&uap->stat->version, &version, sizeof(version)))
        != 0)
        return (error);
    if (version != sizeof(struct kld_file_stat_1) &&
        version != sizeof(struct kld_file_stat))
        return (EINVAL);

    error = kern_kldstat(td, uap->fileid, &stat); ← doesn't full init stat struct
    if (error != 0)
        return (error);
    return (copyout(&stat, uap->stat, version)); ← uninit stat struct copied to user
}

```

Sample bug 2

- kldstat system call
 - FreeBSD 11.0
 - Been there for almost 10 years (Modified *Mon Oct 22 04:12:57 2007 UTC* (9 years, 9 months ago))
 - Doesn't fully initialize a structure. Sends it back to userland
 - infoleak
-
- Previous assumption is not true: *bugs in syscalls do occur with some frequency*
 - Especially newly added syscalls



TCP/IP stack

Attack surface entryptpoint

- TCP/IP stack
 - Ipv4/6
 - Udp/tcp/icmp
 - Ipsec
 - ...
- Obvious and well known attack surface
- Has been around forever
- Assumption: well tested and less likely to find bugs there

```

static void pppoe_dispatch_disc_pkt(struct mbuf *m, int off)
{
...
    int noff, err, errortag;
    u_int16_t *max_payload;
    u_int16_t tag, len;
...
    while (off + sizeof(*pt) <= m->m_pkthdr.len) {
        n = m_pulldown(m, off, sizeof(*pt), &noff);
...
        pt = (struct pppoetag *) (mtod(n, caddr_t) + noff);
        tag = ntohs(pt->tag);
        len = ntohs(pt->len);
...
        switch (tag) {
...
        case PPPOE_TAG_SNAME_ERR:
            err_msg = "SERVICE NAME ERROR";
            errortag = 1;
            break;
...
        }
        if (err_msg) {
            log(LOG_INFO, "%s: %s: ", devname, err_msg);
            if (errortag && len) {
                n = m_pulldown(m, off, len,
                    &noff); ← if m_pulldown() fails, it will mfreem(m)
                if (n) {
                    u_int8_t *et = mtod(n, caddr_t) + noff;
                    while (len--)
                        addlog("%c", *et++);
                }
                } ← should have else case that sets m to NULL
            }
            addlog("\n");
            goto done; ← will end up mfreem(m) again
        }
        off += len;
    }
...
done:
    m_freem(m); ← possible double free!
}

```

Sample bug

- pppoe_dispatch_disc_pkt()
- Double free when parsing packet
- Affects OpenBSD 6.1
 - Been there since 2004
 - Fixed in NetBSD a couple of months ago
- Previous assumption is not [entirely] true: bugs in TCP/IP stack do occur with some frequency
 - newer code
 - mbuf handling is complicated and error prone



Drivers

Attack surface entryptpoint

- Lots and lots of drivers
- For all sorts of things
- UNIX: everything is a file
 - Most expose entryptpoints in /dev
- File operations
 - Open
 - loctl
 - Read
 - Write
 - Close
 - ...
- loctl is where most of the attack surface is!


```
int
cryptof_ioctl(struct file *fp, u_long cmd, void *data)
{
...
    switch (cmd) {
...
        mutex_enter(&crypto_mtx);
        fcr->mtime = fcr->atime;
        mutex_exit(&crypto_mtx);
        mkop = (struct crypt_mkop *)data;
        knop = kmem_alloc((mkop->count * sizeof(struct crypt_n_kop)),
            KM_SLEEP);
        error = copyin(mkop->reqs, knop,
            (mkop->count * sizeof(struct crypt_n_kop)));
        if (!error) {
            error = cryptodev_mkey(fcr, knop, mkop->count);
            if (!error)
                error = copyout(knop, mkop->reqs,
                    (mkop->count * sizeof(struct crypt_n_kop)));
        }
        kmem_free(knop, (mkop->count * sizeof(struct crypt_n_kop)));
        break;
...
    }
}
```



Integer overflow



Memory corruption
due to int overflow

Sample bug

- Crypto device CIOCNFKEYM ioctl
- NetBSD 7.1
 - Been there since NetBSD 4.0.1? *Thu Apr 10 22:48:42 2008*
- Classic integer overflow → memory corruption

```
before ioctl
after ioctl
r: -1
i: 51
ptr: 0xbb912000
before ioctl
uvm_fault(0xc10308c0, 0xc191c000, 1) -> 0xe
fatal page fault in supervisor mode
trap type 6 code 0 eip c02a0c35 cs 8 eflags 10206 cr2 c191c030 ilevel 0 esp bb91
2000
curlwp 0xc1cd7d20 pid 453 lid 1 lowest kstack 0xcde362c0
panic: trap
cpu0: Begin traceback...
vpanic(c0dcea3f,cde37d5c,cde37dd8,c096fcba,c0dcea3f,cde37de4,cde37de4,1,cde362c0
,10206) at netbsd:vpanic+0x121
snprintf(c0dcea3f,cde37de4,cde37de4,1,cde362c0,10206,c191c030,0,bb912000,c1bd81b
0) at netbsd:snprintf
trap_tss() at netbsd:trap_tss
--- trap via task gate ---
0:
cpu0: End traceback...
dump to dev 0,1 not possible
rebooting...
```

```

static int
ksyms_open(st
{
...
    struc
...
    sc = (
        M_
...
    sc->s
    sc->s
...
    error
...
}

```

```

static int
ksyms_mmap(struct cdev *dev, vm_offset_t offset, vm_paddr_t *paddr,
            int prot __unused, vm_memattr_t *memattr __unused)
{
    struct ksyms_softc *sc;
    int error;

    error = devfs_get_cdevpriv((void **)&sc);
    if (error)
        return (error);

    /*
     * XXX mmap() will actually map the symbol table into the process
     * address space again.
     */
    if (offset > round_page(sc->sc_usize) ||
        (*paddr = pmap_extract(sc->sc_pmap, ← can be expired pointer!
                               (vm_offset_t)sc->sc_uaddr + offset)) == 0)
        return (-1);

    return (0);
}

```

Sample bug 2

- Ksyms device
- FreeBSD 11
 - Been there since FreeBSD 8.0 *Tue May 26 21:58:11 UTC 2014*
- Expired pointer
 - open() callback saves pointer to pmap table
 - mmap() callback uses saved pointer in pmap table
 - So how is this a problem ?
 - What if we hand fd off to another process (e.g. kld) (see slide 10)
 - And then we exit
 - If other process now does mmap, it will use the expired pointer

```
Unread portion of the kernel message buffer:

Fatal trap 12: page fault while in kernel mode
cpuid = 0; apic id = 00
fault virtual address = 0xffffffff00f000e000
fault code           = supervisor read data, page not present
instruction pointer   = 0x20:0xffffffff80f8f2fe
stack pointer         = 0x28:0xffffffff004f1da6c0
frame pointer         = 0x28:0xffffffff004f1da700
code segment          = base 0x0, limit 0xfffff, type 0x1b
                     = DPL 0, pres 1, long 1, def32 0, gran 1
processor eflags      = interrupt enabled, resume, IOPL = 0
current process       = 62550 (ksyms_serv)
trap number           = 12
panic: page fault
cpuid = 0
KDB: stack backtrace:
#0 0xffffffff80b24077 at kdb_backtrace+0x67
#1 0xffffffff80ad93e2 at vpanic+0x182
#2 0xffffffff80ad9253 at panic+0x43
#3 0xffffffff80fa0d31 at trap_fatal+0x351
#4 0xffffffff80fa0f23 at trap_pfault+0x1e3
#5 0xffffffff80fa04cc at trap+0x26c
#6 0xffffffff80f84141 at calltrap+0x8
#7 0xffffffff8221c80c at ksyms_mmap+0x4c
#8 0xffffffff80e00cf5 at old_dev_pager_ctor+0x65
#9 0xffffffff80e00286 at cdev_pager_allocate+0x66
#10 0xffffffff80dffd8d at dev_pager_alloc+0x28
#11 0xffffffff80e214e6 at vm_mmap_cdev+0x96
#12 0xffffffff809867c5 at devfs_mmap_f+0x135
#13 0xffffffff80e1f06c at sys_mmap+0x40c
#14 0xffffffff80fa168e at amd64_syscall+0x4ce
#15 0xffffffff80f8442b at Xfast_syscall+0xf
Uptime: 9d1h6m11s
Dumping 237 out of 991 MB:..7%..14%..21%..34%..41%..54%..61%..74%..81%..95%

Reading symbols from /usr/lib/debug/boot/kernel/uhid.ko.debug...done.
Loaded symbols for /usr/lib/debug/boot/kernel/uhid.ko.debug
Reading symbols from /usr/lib/debug/boot/kernel/ksyms.ko.debug...done.
Loaded symbols for /usr/lib/debug/boot/kernel/ksyms.ko.debug
#0 doadump (textdump=<value optimized out>) at pcpu.h:221
221      pcpu.h: No such file or directory.
      in pcpu.h
(kgdb) █
```

Compat code

Attack surface entryptpoint

- The BSDs have binary compatibility [compat] support for some binaries:
 - Older versions of the OS
 - 32bit versions of a program (on a 64bit version of the OS)
 - Other operating system (e.g. Linux)
- Has to emulate a bunch of stuff (e.g. syscalls)

“The people who rely on the compat layers don't care enough to maintain it. The people who work on the mainline system don't care about the compat layers because they don't use them. The cultures aren't aligned in the same direction. ***Compat layers rot very quickly.***” – Theo De Raadt

```

static int
ti_bind(file_t *fp, int fd, struct svr4_striocctl *ioc, struct lwp *l)
{
...
    struct svr4_strmcmd
...
    if (ioc->len > sizeof(k
        return EIN

    if ((error = copyin(N
        return erro

...
    switch (st->s_family
    case AF_INET:

...
        netaddr_t

...
    }

...
}

```

```

#define SVR4_C_ADDROF(sc) (const void *) (((const char *) (sc)) + (sc)->offs)
...
static void netaddr_to_sockaddr_in
    (struct sockaddr_in *sain, const struct svr4_strmcmd *sc)
{
    const struct svr4_netaddr_in *na;

    na = SVR4_C_ADDROF(sc); ← could point to anywhere in memory
    memset(sain, 0, sizeof(*sain));
    sain->sin_len = sizeof(*sain);
    sain->sin_family = na->family; ← crash or info leak
    sain->sin_port = na->port; ← crash or info leak
    sain->sin_addr.s_addr = na->addr; ← crash or info leak

...
}

```

```

/*
 * Pretend that we have streams...
 * Yes, this is gross.
...
*/

```

Sample bug

- SVR 4 streams compat code
- NetBSD 7.1
 - Been there since NetBSD 1.2 *Thu Apr 11 12:49:13 1996*
- Uses offset that comes from userland
 - Without any validation
- Can read arbitrary(-ish) kernel memory
 - Panic
 - Info leak
- CVS commit message on the bugfix:

Log Message:

Fix some of the multitudinous holes in svr4 streams.

We should never have enabled this by default; it is a minefield.



Trap handlers

Attack surface entryptpoint

- Trap handlers handle some kind of exception or fault
 - Div by zero
 - Syscall
 - Breakpoint
 - Invalid memory access
 - ...
- Some can be triggered by userland, and the kernel has to handle them correctly
- due to their nature, they are ugly and highly architecture specific

Fuzz it!

- what would happen if we execute random instructions?
- Surely a bunch of traps
- them

```
int rfd;

void execute_code(unsigned char *p) {
    int (*fn)();
    fn = p;
    fn();
    return;
}

void fuzz() {
    unsigned char *code = mmap(NULL, lenbuf, PROT_EXEC | PROT_READ | PROT_WRITE, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    while(1) {
        read(rfd, code, lenbuf);
        int pid = fork();
        if (pid == -1) {
            exit(0);
        } else if (pid == 0) {
            execute_code(code);
        } else {
            int status;
            pid_t r;
            r = waitpid(pid, &status, 0);
            if (r == -1) {
                kill(pid, 9);
                sleep(1);
                waitpid(pid, &status, WNOHANG);
            }
        }
    }
}

int main(void) {
    rfd = open("/dev/urandom", O_RDONLY);
    fuzz();
}
```

demo!

Hi

- Xe
- td

IT

GDB is free software, covered by the GNU General Public License, and you are welcome to change it and/or distribute copies of it under certain conditions. Type "show copying" to see the conditions. There is absolutely no warranty for GDB. Type "show warranty" for details. This GDB was configured as "amd64-marcel-freebsd"...

Unread portion of the kernel message buffer:
<6>pid 4202 (infuzz), uid 1001: exited on signal 10 (core dumped)
<6>pid 4203 (infuzz), uid 1001: exited on signal 4 (core dumped)
panic: tdsendsignal(): invalid signal 0
cpuid = 0
KDB: stack backtrace:
#0 0xffffffff80b24077 at kdb_backtrace+0x67
#1 0xffffffff80ad93e2 at vpanic+0x182
#2 0xffffffff80ad9253 at panic+0x43
#3 0xffffffff80ae04f7 at tdsendsignal+0x1007
#4 0xffffffff80adf290 at trapsignal+0x230
#5 0xffffffff80fa0948 at trap+0x6e8
#6 0xffffffff80f84141 at calltrap+0x8
Uptime: 15m47s
Dumping 138 out of 991 MB:..12%..24%..35%..47%..58%..70%..81%..93%

Reading symbols from /usr/lib/debug/boot/kernel/uhid.ko.debug...done.
Loaded symbols for /usr/lib/debug/boot/kernel/uhid.ko.debug
#0 doadump (textdump=<value optimized out>) at pcpu.h:221
221 pcpu.h: No such file or directory.
in pcpu.h

(kgdb) bt
#0 doadump (textdump=<value optimized out>) at pcpu.h:221
#1 0xffffffff80ad8e69 in kern_reboot (howto=260) at /usr/src/sys/kern/kern_shutdown.c:366
#2 0xffffffff80ad941b in vpanic (fmt=<value optimized out>, ap=<value optimized out>) at /usr/src/sys/kern/kern_shutdown.c:759
#3 0xffffffff80ad9253 in panic (fmt=0x0) at /usr/src/sys/kern/kern_shutdown.c:690
#4 0xffffffff80ae04f7 in tdsendsignal (p=<value optimized out>, td=<value optimized out>, sig=<value optimized out>, ksi=<value optimized out>) at /usr/src/sys/kern/kern_sig.c:2104
#5 0xffffffff80adf290 in trapsignal (td=0xffffffff8000d04fa00, ksi=0xffffffffe00003f1968) at /usr/src/sys/kern/kern_sig.c:1988
#6 0xffffffff80fa0948 in trap (frame=0xffffffffe00003flac0) at /usr/src/sys/amd64/amd64/trap.c:611
#7 0xffffffff80f84141 in calltrap () at /usr/src/sys/amd64/amd64/exception.S:236
#8 0x00000000800641007 in ?? ()
Previous frame inner to this frame (corrupt stack?)
Current language: auto; currently minimal
(kgdb)

Copyright 2004 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public License, and you are welcome to change it and/or distribute copies of it under certain conditions. Type "show copying" to see the conditions.

for details.

1)

ent

1

.88%..99%

..done.

Loaded symbols for /usr/lib/debug/boot/kernel/uhid.ko.debug
#0 doadump (textdump=<value optimized out>) at pcpu.h:221
221 pcpu.h: No such file or directory.
in pcpu.h
(kgdb)



File systems

Attack surface entryptpoint

- Filesystem attack surface seems easy enough.
 - Malicious fs image that gets mounted
 - Also do file operations on them once mounted
 - Is certainly attack surface
- However, there is more!
- In recent years all 3 BSDs support fuse
- VFS layer now has to deal with malicious data that comes from userland
 - Before it always came from a trusted file system driver

Attack surface entryptpoint [fuse]

- FBSD/OBSD/NBSD all have different fuse implementations (no shared code whatsoever)
 - NBSD: most complete (allows for the most file operations)
 - FBSD: most controlled arguments passed back and forth (getattr, readdir) less opportunity for consumers to make mistakes, but more parsing/processing in fusefs itself, more potential for bugs in fuse code itself
 - OBSD: minimal functional implementation (compared to the previous two)
- none implement ioctl
- all do:
 - read
 - write
 - readdir
 - getattr
 - setattr
 - ...


```

int
vfs_getcwd_scandi
    char *bufp, struct
{
    int eofflag;
    ...
    struct vat
    ...

    dirbuflen
    if (dirbuflen
        ...

    dirbuf = r
    ...
error = VOP_READDIR(uvp, &uio, p->p_ucred, &eofflag); ← fusefs can provide arbitrary content
...
cpos = dirbuf;
...
for (len = (dirbuflen - uio.uio_resid); len > 0;
    len -= reclen) {
    dp = (struct dirent *)cpos;
    reclen = dp->d_reclen;

    /* Check for malformed directory */
    if (reclen < DIRENT_RECSIZE(1)) {
        error = EINVAL;
        goto out;
    }

    if (dp->d_fileno == fileno) {
        char *bp = *bpp;
        bp -= dp->d_namlen; ← fusefs can lie about d_namlen

        if (bp <= bufp) {
            error = ERANGE;
            goto out;
        }

        memmove(bp, dp->d_name, dp->d_namlen); ← out of bound read.

```

Sample bug

- Unbound malloc and out of bound read (could panic or info leak)
- OpenBSD 6.1
 - Been there since OpenBSD 4.0 *Fri Apr 28 08:34:31 2006*
- getcwd syscall when taking data from fuse / userland

```

static daddr_t
ext2_nodealloccg(struct inode *ip, int cg, daddr_t ipref, int mode)
{
...
    error = bread(ip->i_devvp, fsbtodb(fs,
        fs->e2fs_gd[cg].ext2bgd_i_bitmap),
        (int)fs->e2fs_bsize, NOCRED, &bp); ← read from filesystem
...
    ibp = (char *)bp->b_data;
...
    len = howmany(fs->e2fs->e2fs_ipg - ipref, NBBY);
    loc = memchr(&ibp[start], 0xff, len);
    if (loc == NULL) {
        len = start + 1;
        start = 0;
        loc = memchr(&ibp[start], 0xff, len); ← logic driven by fs data
        if (loc == NULL) {
            printf("cg = %d, ipref = %lld, fs = %s\n",
                cg, (long long)ipref, fs->e2fs_fsmnt);
            panic("ext2fs_nodealloccg: map corrupted"); ← panic driven by fs data
            /* NOTREACHED */
        }
    }
}
...
}

```

Sample bug 2

- panic() driven by filesystem data
- FreeBSD 11
 - Been there since FreeBSD 8.1 *Thu Jan 14 14:30:54 2010*
- Ext2 file system code

Networking (bt, wifi,
irda)

Wifi Attack surface entryptpoint

- Stack itself
 - 802.11 network data
 - Parsing
 - Info leaks
- Wifi drivers
 - Data send by device to host

802.11 stack

- One 802.11 stack for all wifi drivers
- Much easier to maintain
 - Need to fix in only 1 place if bugs are found
- `ieee80211_input()` is main parsing input
 - Called from all wifi drivers

```

ieee80211_eapol_key_input(struct ieee80211com *ic, struct mbuf *m,
    struct ieee80211_node *ni)
{
    struct ifnet *ifp = &ic->ic_if;
    struct ether_header *eh;
    struct ieee80211_eapol_key *key;
...
    eh = mtod(m, struct ether_header *);
...
    if (m->m_len < sizeof(*key) &&
        (m = m_pullup(m, sizeof(*key))) == NULL) { ← guarantees that there are sizeof(struct ieee80211_eapol_key) continuous bytes in the mbuf
...
    }
...
    key = mtod(m, struct ieee80211_eapol_key *);
...
    if (m->m_pkthdr.len < 4 + BE_READ_2(key->len)) ← assume key->len is larger than key->payload
        goto done;

    /* check key data length */
    totlen = sizeof(*key) + BE_READ_2(key->paylen); ← assume key->len is larger than key->payload
    if (m->m_pkthdr.len < totlen || totlen > MCLBYTES)
        goto done;
...
    /* make sure the key data field is contiguous */
    if (m->m_len < totlen && (m = m_pullup(m, totlen)) == NULL) { ← not enough data pulled up if key->len is larger than key->payload!
...
    }
    key = mtod(m, struct ieee80211_eapol_key *);
...
                                ieee80211_rcv_4way_msg3(ic, key, ni); ← can crash in here if not enough data is pulled up.
...
}

```


802.11 Stack sample bug

- mbuf mishandling, leading to crash
 - Doesn't guarantee it pulls up enough mbuf data
- OpenBSD 6.1
 - Bug has been there for almost 9 years
- Parsing EAPOL frames

802.11 Drivers

- Wifi drivers are either PCI or USB
- Do you trust the radio?
 - What if it does get compromised?
- Assume PCI cards cause total compromise (they can do DMA)
 - Well, actually, with IOMMU that's no longer the case ...
- USB is packet based protocol
 - Host USB parsers should be able to parse safely
 - Currently BSD wifi drivers do not do this!
 - Leads to trivial heap smashes

```
void
```

```
/*  
 * A  
 * th  
 */  
void  
atu  
{  
    ...  
    struct ndis_wlan_bssid_ex *bss;  
    struct mbuf *m;  
    int pktlen;  
    ...  
    bss = (struct ndis_wlan_bssid_ex *)buf;  
    ...  
    if (__predict_false(len < sizeof(*bss) + letoh32(bss->ieslen))) ← could int overflow  
        return;  
    ...  
    /* Build a fake beacon frame to let net80211 do all the parsing. */  
    pktlen = sizeof(*wh) + letoh32(bss->ieslen); ← could int overflow  
    if (__predict_false(pktlen > MCLBYTES)) ← signedness issue  
        return;  
    MGETHDR(m, M_DONTWAIT, MT_DATA);  
    if (__predict_false(m == NULL))  
        return;  
    if (pktlen > MHLEN) {  
        MCLGET(m, M_DONTWAIT);  
        if (!(m->m_flags & M_EXT)) {  
            m_free(m);  
            return;  
        }  
    }  
    wh = mtod(m, struct ieee80211_frame *);  
    ...  
    memcpy(&wh[1], (uint8_t *)&bss[1], letoh32(bss->ieslen)); ← memory corruption  
    ...  
}
```

memory corruption

802.11 drivers sample bug

- Wide open attack surface
 - Atmel AT76C50x IEEE 802.11b wireless network device [atu(4)]
 - Atheros USB IEEE 802.11a/b/g/n wireless network device [otus(4)]
 - Realtek RTL8188SU/RTL8192SU USB IEEE 802.11b/g/n wireless network device [rsu(4)]
 - Ralink Technology/MediaTek USB IEEE 802.11a/b/g/n wireless network device [run(4)]
 - Atheros USB IEEE 802.11a/b/g wireless network device [uath(4)]
- Across all BSDs
- They didn't think about the attack surface on this one

miscellaneous

- NULL derefs
 - malloc(len, type, M_NOWAIT/M_CANFAIL)
 - Not checking return value
 - Relatively frequent bug
 - M_WAITOK (== can *never* fail) is a very common case
 - Developers treating M_NOWAIT/M_CANFAIL code as if it was M_WAITOK
- DRM/DRI
 - DRM/DRI code base is part of linux kernel source tree.
 - BSD folks fork it
 - Code quality is about as big a shit sandwich as it is in linux DRM/DRI code base

“ All this linux code that we are importing ... is not going to be reviewed by any of the other OpenBSD kernel developers ... because they refuse to read any code that is not conformant to the BSD KNF standard” – Matthieu Herrb

Results

- results:
 - About ~115 kernel bugs so far
 - FBSD: ~30
 - OBSD: ~25
 - NBSD: ~60
 - types of bugs seen:
 - Straight heap/stack smash
 - race conditions
 - expired pointers
 - Double frees
 - recursion issues
 - integer issues
 - Underflows, overflows, signedness
 - Refcount issues
 - Overflows
 - Reference dropped too soon → use after free
 - info leaks
 - out of bound read
 - NULL deref
 - Logic bugs
 - typo
 - Division by zero
 - kernel panics driven by userland
 - Memory leaks

Conclusions

- Bugs were found in all 3 of the examined BSDs
 - Among all of the attack surfaces mentioned above
- Winner / loser
 - OBSD clear winner (they have massively reduced their attack surface over the years):
 - Attack surface reduction
 - no loadable modules
 - relatively few devices
 - Virtually no compat code (they removed Linux a couple of years ago)
 - removed entire Bluetooth stack
 - Significantly less syscalls (e.g. 200+ syscalls less than FBSD)
 - Cut support for some older architectures
 - Code Quality
 - int overflows / signedness bugs, as good as gone in most places
 - Few info leaks
 - NBSD clear loser
 - Tons of legacy and compat code (who the hell still needs the ISO protocols ??? Really?)
 - seems to be less consistent with security code quality
 - Too many signedness bugs.
 - This is **NOT** a dis.
 - if you think building/maintaining/improving an OS is easy, go ahead, try it. See how far you get.
- FBSD is somewhere in between



Conclusions

- Security team responses
 - OpenBSD
 - About a week or so to get response (Theo said it took so long because he was on vacation)
 - Bugfixes rolled out in the next couple of days
 - FreeBSD
 - Response in about a week or so
 - The filed bugs internally
 - Don't know what the status of those bugs is
 - NetBSD
 - They fixed virtually all bugs submitted. Pretty much overnight !!!
 - That is ridiculously impressive.
 - They also turned off the SVR4 subsystem for i386 by default

“We have also disabled COMPAT_SVR4 by default on i386, something that should have been done a long time ago.” – Response from NetBSD developers

More conclusions

- Bugs *are* still easy to find in those kernels. Even OpenBSD.
- Varying level of quality depending on age and who wrote it
 - Most consistent quality was observed with OpenBSD
- The maintainers of various BSDs should talk more among each other
 - Several bugs in one were fixed in the other
 - OpenBSD expired proc pointer in midioctl() fixed in NetBSD
 - NetBSD signedness bug in ac97_query_devinfo() fixed in OpenBSD

More conclusions

- Code base size
 - OpenBSD: 2863505 loc
 - NetBSD: 7330629 loc
 - FreeBSD: 8997603 loc
- Obviously this plays a part
 - Can't have a bug in code you don't have
- Accidental vs. planned
 - Haven't gotten to implementing something yet or ...
 - Choice made on purpose to delete code
 - Attack surface reduction

More conclusions

- Many eyeballs ...
- Gut feeling, I suspect this is a factor.
- Based on my result, code quality alone can't account for the discrepancy between the bug numbers (BSD vs. Linux).
- Say what you will about the people reviewing the Linux kernel code, there are simply orders of magnitude more of them. And it shows in the numbers.

Questions / comments from the internet

- Defcon releases presentation before you actually perform the presentation
 - This is kind of annoying....
 - People saw it, and commented on it before I had a chance to stand up here ...
 - Surprisingly little hate / trolls (yet?)
 - Got some from the OS zealots
- Why didn't you do the same for linux?
 - The numbers presented earlier speak for themselves IMO. Assumed them (and what they imply) to be accurate
- You conclude that linux is better! And it's not even the subject of the presentation
 - I did not.
- You should've added a comparison of protection mechanisms between linux and the BSDs
 - While that would be interesting, it's far outside of the scope. Doesn't relate to code quality. Also time constraints.
- What about dragonflyBSD? HardenedBSD?
 - I considered it. Maybe I should have too. I had limited amount of time and picked the 3 most commonly used BSDs

Questions / comments from the internet

- Interesting related links I got from the internet
 - *How to find 56 potential vulnerabilities in FreeBSD code in one evening, PVS-Studio delved into the FreeBSD kernel*
 - <https://www.viva64.com/en/b/0496/>
 - <https://www.viva64.com/en/b/0377/>
 - <https://www.viva64.com/en/b/0487/>
 - FreeBSD – A lesson in poor defaults
 - <https://vez.mrsk.me/freebsd-defaults.txt>

Questions ?

That's all Folks!